System Programming Project 3

담당 교수 : 이영민 교수님

이름 : 김윤하

학번 : 20211518

1. **개발 목표**

이번 세 번째 프로젝트의 목표는 여러 클라이언트와 동시(concurrent) 통신이 가능한 stock server를 구현하는 것이다. 이 서버는 *stock.txt* 파일에 저장된 주식 정보를 읽어 이진 탐색 트리(Binary Search Tree) 형태로 메모리에 저장하며, 클라이언트의 명령에 따라 주식 정보를 처리한다. 서버는 두 가지 방식 중 하나로 구현된다:

- **Event-driven 방식** (*select*() 기반)
- **Thread-based 방식** (*pthread* 기반)

서버는 클라이언트로부터 다음 네 가지 명령을 수신하고 처리한다:

1. ***buy [ID] [수량] :*** 특정 주식(ID)을 원하는 수량만큼 구매 요청
    - 서버는 해당 ID의 존재 여부와 충분한 재고를 확인 후:
        - 해당 ID가 없는 경우 "Invalid stock ID" 전송
        - 재고가 충분하면 재고를 차감하고 "[buy] success" 전송
        - 부족하면 "Not enough left stocks" 메시지 전송
2. ***sell [ID] [수량]:*** 클라이언트가 특정 주식을 판매하려는 요청
    - 서버는 해당 주식의 재고를 요청 수량만큼 증가시키고,

    "[sell] success" 메시지를 전송한다.

3. ***show***
    - 현재 모든 주식의 ID, 잔여 재고, 가격 정보를 클라이언트에게 전송
4. ***exit***: 클라이언트 연결 종료 처리

연결된 클라이언트가 모두 종료되거나, 서버가 종료되는 경우에만 기존의 *stock.txt* 파일을 업데이트한다.

2. 개발 범위 및 내용

A. 개발 범위

1. Task 1: Event-driven Approach

   서버가 실행되면서 stock.txt의 정보를 이진 트리에 저장한다.

   이후 pool이 초기화되면서, 클라이언트와의 연결을 총괄한다.

   Pool에는 현재 연결된 모든 클라이언트의 파일 디스크립터(fd)가 저장되어 있다. 서버는 종료되기 전까지, 이 fd들을 통해 클라이언트와 통신을 수행한다. 무한 루프(while(1)) 내에서 select() 함수를 반복 호출하며, pool에 등록된 fd들 중 이벤트가 발생한 것을 감지하고, 해당 클라이언트로부터 입력을 받아 처리한 후 응답을 전송한다.
   이러한 처리는 모두 순차적으로 이루어지므로, 별도의 lock이나 mutex 같은 동기화 장치는 필요하지 않다.

2. Task 2: Thread-based Approach

   앞선 select 기반 서버처럼, phtread기반 서버는 시작 시 stock.txt 파일을 읽어 이진 탐색 트리에 저장한다.

   이후 각 클라이언트가 접속하면 새로운 스레드가 생성되고, thread 함수에서 클라이언트의 show, buy, sell, exit 명령을 처리한다.

   buy나 sell 명령 시 같은 stock 항목에 접근하는 동시성 문제를 방지하기 위해, 각 주식 종목별로 semaphore 기반 mutex lock 를 사용하여 race condition을 방지한다. 클라이언트가 exit 명령을 보내면 연결이 종료되며, 마지막 클라이언트가 나갈 경우 stock.txt를 업데이트한다. 또한 SIGINT 핸들러를 통해, 서버가 종료될 때 stock.txt가 다시 한번 업데이트된다.

3. Task 3: Performance Evaluation

Multiclient 파일에 *gettimeofday* 코드를 추가하여, 전체 수행 시간을 계산한다. 연결이 종료되어 multiclient가 종료되기 직전, *(num_client*ORDER_PER_CLIENT)*을 전체 수행 시간(elapsed time)으로 나누어 동시 처리율(시간당 client 처리 요청 개수)을 구한다. 수행 명령어를 하나로 고정하여, 같은 명령어에 대해서 서로 다른 방식의 서버의 처리율을 비교할 수 있다.

B. 개발 내용

- **Task1 (Event-driven Approach with select())**

이 서버는 *select* 시스템 콜을 이용해 다수의 클라이언트로부터 동시에 요청을 처리하는 이벤트 기반 서버(Event-driven server) 구조로 구성되어 있다. 서버 실행 시, stock.txt에 저장된 데이터를 이진 탐색 트리에 저장한 후, 클라이언트 요청에 따라 메모리 상에서 show, buy, sell과 같은 주식 거래 명령을 처리한다. 이진 탐색 트리를 사용했기 때문에, O(logN) 의 시간 복잡도를 가진다.

무한루프 속에서 계속 select 함수를 통해 여러 소켓 fd를 검사하여 이벤트 발생 여부를 탐지한다. 이를 통해 I/O Multiplexing 방식으로 여러 클라이언트의 요청을 동시에 처리할 수 있게 된다. 모든 클라이언트 연결은 하나의 FD_SET에 등록되고, 루프 내에서 select를 통해 어떤 소켓에 데이터가 도착했는지를 검사한 후 해당 소켓에만 read 및 write 처리를 수행한다.

하지만 select()는 검사할 수 있는 fd의 수가 제한적이며(maxfd), 매번 모든 fd를 검사하기 때문에 이벤트가 발생하지 fd도 반복적으로 확인하는 오버헤드가 존재한다. 이에 비해 *epoll*은 이벤트가 발생한 fd만을 알려주기 때문에 확장성과 성능 측면에서 select보다 우수하다. 특히 epoll은 fd의 수가 많거나 많은 이벤트가 발생하는 상황에서 select보다 훨씬 우수한 성능을 낸다.

- **Task2 (Thread-based Approach with pthread)**

본 서버에서는 Master Thread가 클라이언트의 연결을 `Accept()`로 받아 처리한다. 클라이언트로부터 연결 요청이 들어오면, 소켓 디스크립터를 동적으로 할당하고, 이를 인자로 하여 새로운 스레드(*pthread*)를 생성한다. 이 방식은 각 클라이언트 연결마다 별도의 스레드를 생성하여, 여러 클라이언트의 요청을 동시에 처리할 수 있도록 한다.

Master Thread는 연결 수락 이후 스레드 생성만을 담당하며, 클라이언트 요청 처리는 생성된 개별 thread에 위임된다. 클라이언트가 `exit` 명령을 보내거나 접속을 종료하면 break문을 통해 해당 스레드는 자동으로 자원을 반납하고 종료된다.

또한 Worker Thread Pool을 미리 생성하지 않고, 클라이언트가 새로 연결될 때마다 동적으로 새 스레드를 생성한다. 생성된 각 스레드는 클라이언트의 명령어를 입력 받아 서버의 이진 탐색 트리에 저장된 주식 정보에 접근하여

`show`, `buy`, `sell` 명령을 수행한다.

앞선 Task1에서와 달리, thread 기반 concurrency에서는 data race가 발생할 가능성**이 있으므로, 각 stock 항목에 대해 semaphore(mutex)를 사용하여 동기화를 구현하였다. 특히 buy, sell처럼 데이터를 수정하는 연산 시에는 P와 V 연산을 통해 mutex lock을 획득 및 해제하며, 동시에 여러 스레드가 동일한 자원에 접근해 생기는 충돌을 방지한다.

또한, 전역 변수 *thread_count*를 통해 현재 연결 중인 클라이언트 수를 관리하며, 연결된 모든 스레드가 종료될 시 `stock.txt` 파일을 업데이트하여 데이터 손실을 방지할 수 있도록 설계하였다. SIGINT 시그널 핸들러를 구현하여, 서버 종료 시 stock.txt를 업데이트 한다.

- **Task3 (Performance Evaluation)**

*gettimeofday*함수를 multiclient.c에 추가하여, 연결이 시작되기 전부터 연결이 모두 해제 되고 나서 multiclient 프로그램이 종료되긴 전까지의 시간 간격을 계산한다. 이후 전체 명령어의 개수는 각 클라이언트의 명령어 수와 전체 클라이언트의 수를 곱해서 구하고, 이를 앞서 구한 시간 간격으로 나누어 초당 몇 개의 명령어를 처리하였는지 처리율을 구한다. 이를 위해 multiclient의 usleep함수를 주석 처리하였다.

각 클라이언트당 항상 10개의 명령어를 처리하도록 고정시킨 채, 전체 클라이언트의 개수를 1개, 10개, 20개, 40개, 80개씩 서로 다른 환경에서 수행 시간(elapsed_sec)과 처리율 : {ORDER_PER_CLIENT}* {num_client}/ {elapsed_sec}을 구하였다.

여러 명령들이 서버에 의해 동시에 수행될 수 있으므로, 전체 클라이언트의 개수의 증가에 따라 전체 명령어의 수가 급증하더라도 전체 수행 시간 증가율이 선형적으로 증가하지 않을 것으로 예상된다. 즉, 많은 명령어를 동시에 처리해 클라이언트의 수가 많아질수록 처리율 또한 같이 높아질 것으로 예상된다.

이를 확인하기 위해 명령어를 buy, sell, show 중 하나로 고정시켰다. 여러 명령어가 섞인 경우도 고려하였으나, 결국엔 각 클라이언트마다 rand 함수로 랜덤하게 어느 명령어 중 하나를 수행하는 것이니, event-based와 thread-based를 직접적으로 비교하기에는 적합하지 않다는 생각이 들었다. 그래서 multiclient.c의 option을 0,1,2 중 하나로 고정하여, 특정 명령어만 수행하도록 하여 buy, sell, show 만을 수행할 시 동시처리율이 어떻게 변하는지 확인하고자 하였다.

우선 클라이언트의 수가 많아질수록 어느 정도 동시처리율이 증가하겠지만, 이후

에는 그 증가율이 줄어들 것 같다고 예상하였다. 아무래도 cspro에 접속해서 프로그램을 돌리는 것이다 보니, cspro의 성능 한계치 수준에서 sweet spot를 가질 것 같다.

또한 event기반에서와 달리, thread 기반에서는 전체 fd를 조사하지 않고 살아있는 쓰레드만이 작동하기 때문에, 더 높은 처리율을 가지며 클라이언트의 수가 늘어날수록 동시처리율의 증가율이 이벤트 기반 서버보다 더 높을 것으로 예상했다.

Buy과 달리 sell 명령어에서는 주식의 잔여 수량을 확인하지 않으므로 buy보다 sell리 더 높은 동시처리율을 가질 것이며, show 명령어에서는 전체 주식 정보를 계속 출력해야 하므로 다른 명령어보다 낮은 동시처리율을 가질 것이라 예상했다.

C. 개발 방법

-Task1 – Event based Server

주식 정보를 담고 있는 stock_item 구조체를 정의하였다

```
typedef struct{
    int ID;
    int left_stock;
    int price;
    int readcnt;
    sem_t mutex;
}stock_item;
```

주식 정보(data)를 담고 있는 Node 구조체를 정의하였다.

```
typedef struct Node{
    stock_item data;
    struct Node* left;
    struct Node* right;
}Node;
```

이벤트 기반 서버에서 사용되는 pool을 초기화하는 함수를 정의하였다.

```
void init_pool(int listenfd, pool *p); /* Initializes pool */
```

또한 `void insert_Node, Node* ROOT`

를 통해 주식 정보를 이진 트리 형태로 전역 노드 변수 ROOT에 저장하고,

`void free_Node`,를 통해 함수 종료 시 메모리 할당을 해제해주었다.

```
stock_item* find_stock(Node* root, int ID);
```

find_stock을 통해 주식ID에 해당하는 주식 정보의 위치를 찾는다.

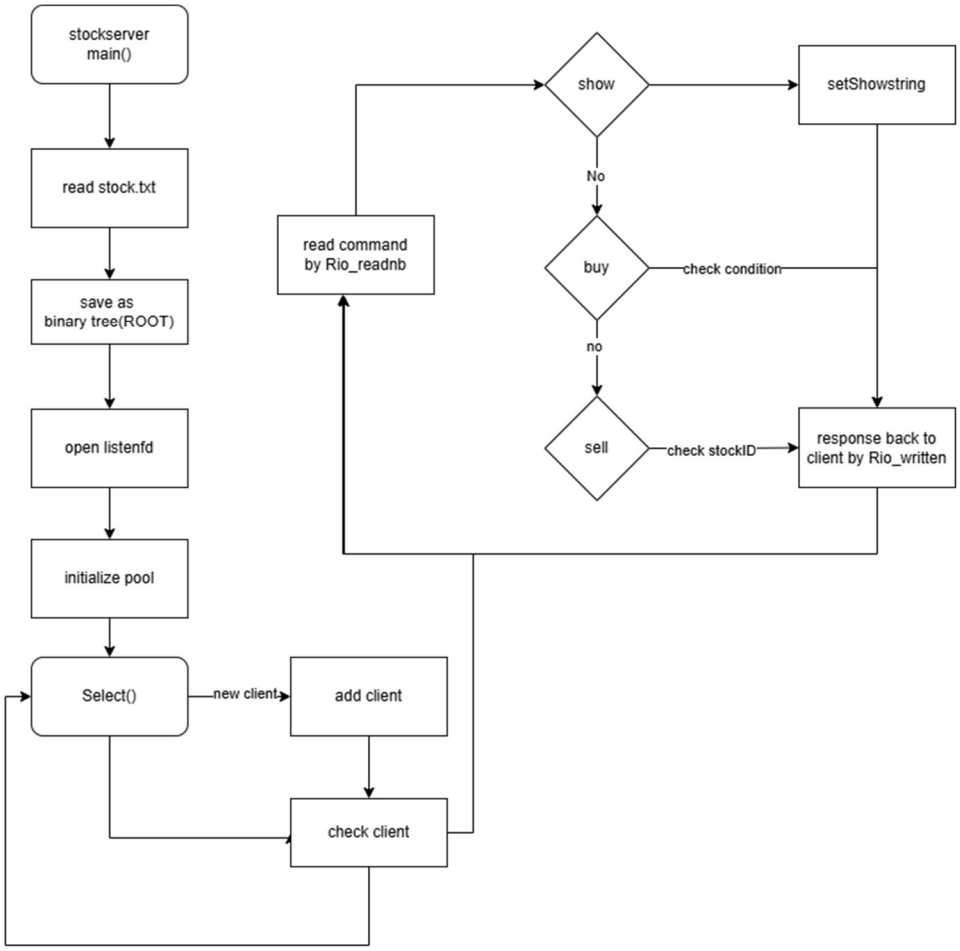
sigint핸들러나, 전체 connection의 연결이 끊긴 경우에 stock.txt를 업데이트한다.

먼저 서버 실행시 init_pool(int listenfd, pool *p) 함수가 호출되어 서버의 클라이언트 연결 관리용 풀(pool)을 초기화한다. 서버는 listenfd (통해 클라이언트 연결 요청을 기다린다. 클라이언트가 접속하면 add_client(int connfd, pool *p) 함수가 호출되어 새 클라이언트를 pool에 추가한다. 서버는 check_clients(pool *p) 함수에서 여러 클라이언트 소켓을 확인한다. 클라이언트가 보낸 요청을 감지하면 해당 클라이언트 소켓으로부터 데이터를 읽고, 명령어에 따라 처리한다.

Show 명령어 수행시에는 빈 문자열 showString에 이진 트리에 담긴 주식 데이터를 문자열로 추가하여, 모든 주식의 정보를 클라이언트에게 한번에 전송한다.

Buy, sell 명령어 수행시에는 해당 주식ID가 존재하는지 확인하며, buy 경우에만 잔여 재고와 구매량을 비교하여 명령 실행 가능 여부를 판단한다.

이처럼 서버는 클라이언트로부터 입력받은 명령어에 따른 명령을 수행하며, 수행 결과를 클라이언트에 전송하고 다른 fd로 넘어가 요청이 들어왔는지 확인하면서 여러 명령들을 concurrent하게 수행한다.



<Event based 서버의 Flowchart>

-Task2: Thread based Server

스레드 기반 서버는 이벤트 기반 서버와 유사하나, select대신 쓰레드를 생성하고, mutex lock을 사용한다는 점이 있다. 앞선 서버처럼 이진 탐색 트리에 주식 데이터를 저장하지만, 이벤트와 달리 구조체 stock_item에 담긴 세마포어(mutex)를 포함한다.

서버는 master thread가 클라이언트 연결 요청을 받아들이고, 연결된 클라이언트 소켓 디스크립터를 버퍼에 저장한다. 이후 worker thread pool이 버퍼에서 소켓을 꺼내 클라이언트 요청을 처리한다. worker thread는 총 10개, 버퍼 크기는 50개로 설정되어 있다. 생성된 worker thread는 클라이언트로부터 명령을 받아 서버로 전달한다.

- show 명령은 이진 트리의 모든 노드 정보를 출력한다. 이때 각 노드의 readcnt 값을 *mutex*로 보호하며 증가시킨 후, 만약 readcnt가 1이 되면 해당 노드에 대한 다른 접근을 차단하여 다중 reader에 의한 동시 접근을 안전하게 관리한다. 출력이 끝난 후에는 readcnt를 감소시키고 0이 되면 다시 접근을 허용한다. 이로써 다수의 reader가 동시에 접근할 수 있지만, buy, sell과 같이 writer가 접근할 경우에는 상호 배제를 보장한다.
- buy 명령은 입력받은 주식 ID에 대해 이진 트리에서 노드를 찾고, 해당 노드의 mutex를 잠가 잔여 주식 수량을 검사한다. 만약 구매 수량이 잔여 수량보다 많으면 실패 메시지를 보내고 *mutex*를 해제한다. 그렇지 않으면 잔여 수량을 감소시키고 성공 메시지를 보낸다.
- sell 명령도 buy와 유사하게 노드를 찾아 *mutex*로 보호하며 잔여 수량을 증가시킨다.

이진 트리의 각 노드에 대해 별도의 *mutex semaphore*를 통해 *lock*을 구현했다. 서버는 클라이언트가 연결 종료(exit)를 요청하거나 연결이 끊길 때까지 명령을 처리하며, 모든 클라이언트의 연결이 종료되거나 서버가 종료되면 현재 트리 상태를 파일(stock.txt)에 저장한다.

3, 구현 결과

서버와 클라이언트가 정상적으로 작동하는 모습을 볼 수 있었다.

```
Server received 5 (777 total) bytes on fd 6
Server received 5 (782 total) bytes on fd 7
Server received 5 (787 total) bytes on fd 8
Server received 8 (795 total) bytes on fd 9
Server received 8 (803 total) bytes on fd 10
Server received 9 (812 total) bytes on fd 11
Server received 8 (820 total) bytes on fd 12
Server received 5 (825 total) bytes on fd 13
All clients disconnected.
'stock.txt' has been successfully updated.
^C
Terminating Server . . .
Server has been successfully terminated
'stock.txt' has been successfully updated.
cse20211518@cspro9:~/project3/20211518/task_1$
```

```
7 1288 7300
2 1434 500
1 1481 1200
4 1435 4600
3 1375 2500
6 1476 9100
5 1328 8000
10 1475 10000
9 1234 4000
8 1246 9900
[buy] success
[buy] success
[sell] success
[buy] success
7 1288 7300
2 1440 500
1 1481 1200
4 1425 4600
3 1375 2500
6 1476 9100
5 1328 8000
10 1475 10000
9 1234 4000
8 1245 9900
cse20211518@cspro9
```

다만 cspro 서버의 상황에 따라, 같은 client의 수를 연결하더라도 어떤 때는 출력이 바로 나오고, 어떤 때는 몇 초 후에 출력이 나오는 경우가 있었다.

4. 성능 평가 결과 (Task 3)

앞서 말한 대로, 성능을 평가하기 위해 총 30번의 수행을 하였다.

1) 각 클라이언트의 수를 1, 10, 20, 40 80 개로

2) 명령어를 buy로 고정, sell로 고정, show로 고정

수행 결과를 스크린샷으로 저장하였고, 엑셀을 이용하여 꺾은선그래프로 나타내었다.

다음은 총 30번의 수행 결과이다:

1) BUY_EVENT의 경우

```
[buy] success
✅ Total clients: 1
🕒 Total time: 5061 microseconds (0.005061 seconds)
📦 Total orders: 10
⚡ 처리율 (Throughput): 1975.89 requests/sec
```

```
[buy] success
[buy] success
✅ Total clients: 10
🕒 Total time: 8338 microseconds (0.008338 seconds)
📦 Total orders: 100
⚡ 처리율 (Throughput): 11993.28 requests/sec
```

```
[buy] success
✅ Total clients: 20
🕒 Total time: 14544 microseconds (0.014544 seconds)
📦 Total orders: 200
⚡ 처리율 (Throughput): 13751.38 requests/sec
```

```
[buy] success
[buy] success
✅ Total clients: 40
🕒 Total time: 25698 microseconds (0.025698 seconds)
📦 Total orders: 400
⚡ 처리율 (Throughput): 15565.41 requests/sec
```

```
[buy] success
[buy] success
[buy] success
✅ Total clients: 80
🕒 Total time: 45212 microseconds (0.045212 seconds)
📦 Total orders: 800
⚡ 처리율 (Throughput): 17694.42 requests/sec
```

2) BUY_Thread의 경우

```
[buy] success
[buy] success
✅ Total clients: 1
🕒 Total time: 1989 microseconds (0.001989 seconds)
📦 Total orders: 10
⚡ 처리율 (Throughput): 5027.65 requests/sec
```

```
Not enough left stocks
[buy] success
Not enough left stocks
✅ Total clients: 10
🕒 Total time: 3440 microseconds (0.003440 seconds)
📦 Total orders: 100
⚡ 처리율 (Throughput): 29069.77 requests/sec
```

```
[buy] success
✅ Total clients: 20
🕒 Total time: 6066 microseconds (0.006066 seconds)
📦 Total orders: 200
⚡ 처리율 (Throughput): 32970.66 requests/sec
```

```
Not enough left stocks
✅ Total clients: 40
🕒 Total time: 9566 microseconds (0.009566 seconds)
📦 Total orders: 400
⚡ 처리율 (Throughput): 41814.76 requests/sec
```

```
Not enough left stocks
✅ Total clients: 80
🕒 Total time: 18409 microseconds (0.018409 seconds)
📦 Total orders: 800
⚡ 처리율 (Throughput): 43457.00 requests/sec
```

3)SELL_EVENT의 경우

```
[sell] success
[sell] success
✅ Total clients: 1
🕒 Total time: 4892 microseconds (0.004892 seconds)
📦 Total orders: 10
⚡ 처리율 (Throughput): 2044.15 requests/sec
```

```
[sell] success
[sell] success
[sell] success
✅ Total clients: 10
🕒 Total time: 8085 microseconds (0.008085 seconds)
📦 Total orders: 100
⚡ 처리율 (Throughput): 12368.58 requests/sec
```

```
[sell] success
[sell] success
[sell] success
[sell] success
✅ Total clients: 20
🕒 Total time: 14106 microseconds (0.014106 seconds)
📦 Total orders: 200
⚡ 처리율 (Throughput): 14178.36 requests/sec
```

```
[sell] success
[sell] success
[sell] success
✅ Total clients: 40
🕒 Total time: 24272 microseconds (0.024272 seconds)
📦 Total orders: 400
⚡ 처리율 (Throughput): 16479.89 requests/sec
```

```
[sell] success
✅ Total clients: 80
🕒 Total time: 41324 microseconds (0.041324 seconds)
📦 Total orders: 800
⚡ 처리율 (Throughput): 19359.21 requests/sec
```

4)SELL_Thread의 경우

```
[sell] success
[sell] success
✅ Total clients: 1
🕒 Total time: 1767 microseconds (0.001767 seconds)
📦 Total orders: 10
⚡ 처리율 (Throughput): 5659.31 requests/sec
```

```
[sell] success
[sell] success
✅ Total clients: 10
🕒 Total time: 3475 microseconds (0.003475 seconds)
📦 Total orders: 100
⚡ 처리율 (Throughput): 28776.98 requests/sec
```

```
[sell] success
[sell] success
✅ Total clients: 20
🕒 Total time: 6022 microseconds (0.006022 seconds)
📦 Total orders: 200
⚡ 처리율 (Throughput): 33211.56 requests/sec
```

```
[sell] success
[sell] success
✅ Total clients: 40
🕒 Total time: 9912 microseconds (0.009912 seconds)
📦 Total orders: 400
⚡ 처리율 (Throughput): 40355.13 requests/sec
```

```
[sell] success
[sell] success
✅ Total clients: 80
🕒 Total time: 16579 microseconds (0.016579 seconds)
📦 Total orders: 800
⚡ 처리율 (Throughput): 48253.82 requests/sec
```

5)SHOW_Thread의 경우

```
8 1254 9900
✅ Total clients: 1
🕒 Total time: 5534 microseconds (0.005534 seconds)
📦 Total orders: 10
⚡ 처리율 (Throughput): 1807.01 requests/sec
```

```
8 1254 9900
✅ Total clients: 10
🕒 Total time: 8412 microseconds (0.008412 seconds)
📦 Total orders: 100
⚡ 처리율 (Throughput): 11887.78 requests/sec
```

```
8 1254 9900
✅ Total clients: 20
🕒 Total time: 15787 microseconds (0.015787 seconds)
📦 Total orders: 200
⚡ 처리율 (Throughput): 12668.65 requests/sec
```

```
8 1254 9900
✅ Total clients: 40
🕒 Total time: 25762 microseconds (0.025762 seconds)
📦 Total orders: 400
⚡ 처리율 (Throughput): 15526.74 requests/sec
```

```
8 1254 9900
✅ Total clients: 80
🕒 Total time: 45336 microseconds (0.045336 seconds)
📦 Total orders: 800
⚡ 처리율 (Throughput): 17646.02 requests/sec
```

6)SHOW_Thread의 경우

```
8 893 9900
✅ Total clients: 1
🕒 Total time: 1812 microseconds (0.001812 seconds)
📦 Total orders: 10
⚡ 처리율 (Throughput): 5518.76 requests/sec
```

```
8 893 9900
✅ Total clients: 10
🕒 Total time: 3918 microseconds (0.003918 seconds)
📦 Total orders: 100
⚡ 처리율 (Throughput): 25523.23 requests/sec
```

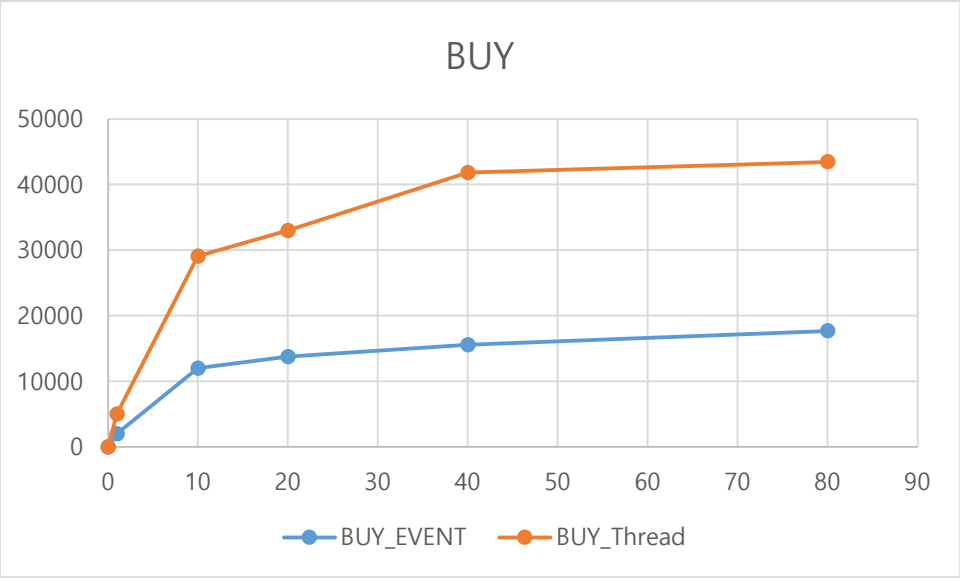
```
8 893 9900
✅ Total clients: 20
🕒 Total time: 7141 microseconds (0.007141 seconds)
📦 Total orders: 200
⚡ 처리율 (Throughput): 28007.28 requests/sec
```

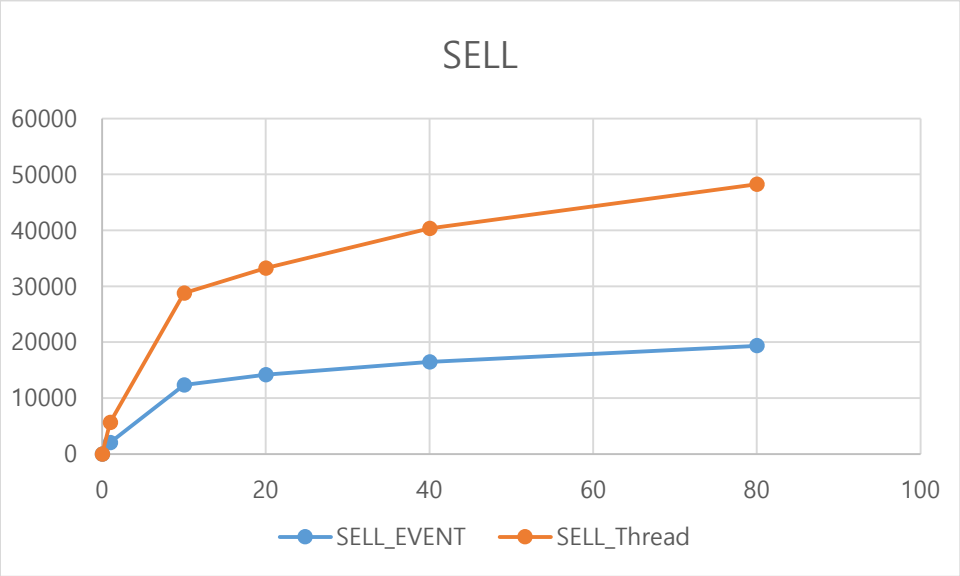
```
8 893 9900
✅ Total clients: 40
🕒 Total time: 12826 microseconds (0.012826 seconds)
📦 Total orders: 400
⚡ 처리율 (Throughput): 31186.65 requests/sec
```

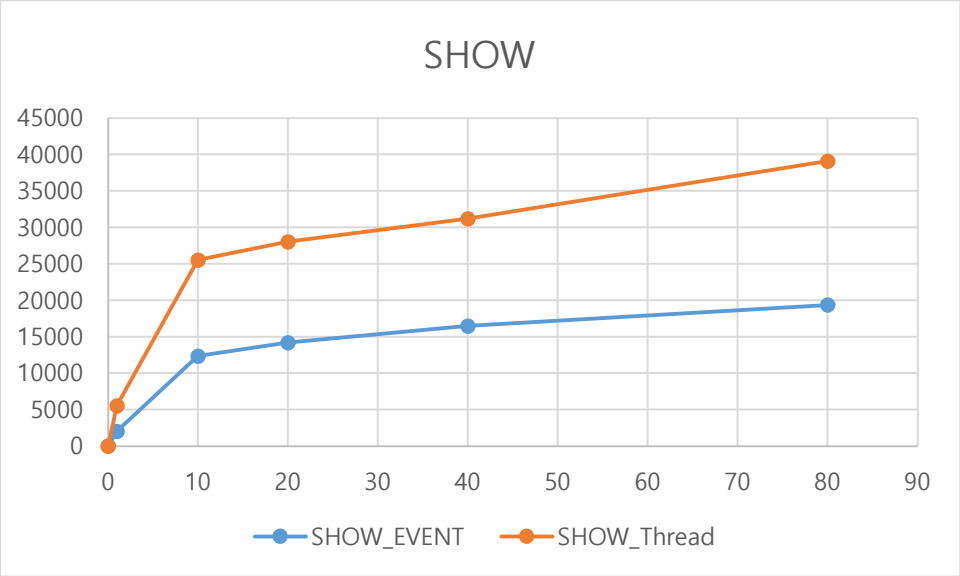
```
8 893 9900
✅ Total clients: 80
🕒 Total time: 20458 microseconds (0.020458 seconds)
📦 Total orders: 800
⚡ 처리율 (Throughput): 39104.51 requests/sec
```

여기까지의 수행 결과를 다음 페이지에 그래프로 나타내었다.

모든 경우에 대해서 Thread 기반 서버가 우세한 결과를 보였으며, 클라이언트 수가 늘어날수록 큰 효율 차를 보였다.


BUY
50000
40000
30000
20000
10000
0
0
10
20
30
40
50
60
70
80
90
BUY_EVENT
BUY_Thread


SELL
60000
50000
40000
30000
20000
10000
0
0
20
40
60
80
100
SELL_EVENT
SELL_Thread


SHOW
45000
40000
35000
30000
25000
20000
15000
10000
5000
0
0
10
20
30
40
50
60
70
80
90
SHOW_EVENT
SHOW_Thread

예상했던 대로, 모든 실험에서 select 기반 이벤트 서버가 pthread 기반 스레드 서버보다 더 높은 동시 처리율(throughput)을 보였다. 클라이언트 수가 증가함에 따라 동시 처리율도 급격히 증가하다가, 일정 시점 이후에는 증가폭이 점차 줄어드는 형태를 보였다. 이는 cspro 서버의 기능적 한계와 내가 작성한 코드의 효율성 한계로 인해, 클라이언트 수가 20 명을 초과하면서부터는 처리 효율이 급격히 상승하지 않고 완만하게 증가했기 때문이다.

또한, 클라이언트 수가 많아질수록 스레드 서버의 처리량 증가폭이 이벤트 서버보다 크다는 점도 확인할 수 있었다. 이는 이벤트 기반 방식의 단점인 "maxfd 까지의 모든 fd 를 매번 확인해야 하는 부담"보다, 스레드 방식의 단점인 "context switch 오버헤드"가 실험 환경에서는 적은 영향을 주었기 때문으로 보인다. 실제로는 모든 실험에서 이벤트 기반 서버가 스레드 기반 서버보다 더 긴 elapsed time 과 낮은 효율을 보였다. 만약 select 대신 epoll 이나 poll 을 사용했다면, 보다 나은 성능을 기대할 수 있었을 것이다.

실험에서는 간단하게 주식 ID 를 검색해 수량을 증감시키는 buy/sell 연산만 수행했기 때문에, 스레드 기반 서버에서 예상보다 적은 context switching 오버헤드가 발생한 것으로 보인다. 또한 *stock.txt* 에는 단 10 개의 주식 정보만 담겨 있어, 데이터의 양이 적고 연산이 단순한 상황에서는 스레드 방식이 오히려 더 나은 성능을 보였다. 만약 더 많은 데이터와 복잡한 연산이 포함되었다면, context switching 오버헤드가 커지면서 이벤트 기반 방식이 더 유리했을 가능성도 있다.

Show 의 경우, 앞선 buy/sell 연산에 비해 상대적으로 효율 차이가 크지 않았다. 이는 연산 과정이 전혀 없이, 단순히 서버에 이진 탐색 트리 형태로 저장된 정보를 출력만 하기 때문에 발생한 결과로 보인다.

마지막으로, 수업 시간에 다뤘듯이 이벤트 기반 서버의 구현이 스레드 기반 서버에 비해 훨씬 단순하고 구현 시간이 짧았다. 상대적으로 적은 시간과 노력으로도 동작 가능한 서버를 만들 수 있었다는 점에서 구현의 용이성 측면에서는 이벤트 기반 서버가 유리했다.